

Complejidad Temporal en Python

La complejidad temporal es una medida de la cantidad de tiempo que toma un algoritmo para ejecutarse en función del tamaño de la entrada. En Python, se puede analizar la complejidad temporal de diferentes estructuras de datos y algoritmos utilizando la notación Big O.

Notación Big O

La notación Big O describe el peor caso de la complejidad temporal de un algoritmo. Aquí hay algunas complejidades comunes:

- **O(1)**: Tiempo constante. El tiempo de ejecución no cambia con el tamaño de la entrada.
- **O(log n)**: Tiempo logarítmico. El tiempo de ejecución crece logarítmicamente con el tamaño de la entrada.
- **O(n)**: Tiempo lineal. El tiempo de ejecución crece linealmente con el tamaño de la entrada.
- **O(n log n)**: Tiempo linealítmico. Común en algoritmos de ordenamiento eficientes como Merge Sort.
- **O(n²)**: Tiempo cuadrático. El tiempo de ejecución crece cuadráticamente con el tamaño de la entrada. Común en algoritmos de ordenamiento menos eficientes como Bubble Sort.

Ejemplos en Python

Tiempo Constante - O(1)

```
def acceso_constante(lista, indice):  
    return lista[indice]
```

Tiempo Lineal - O(n)

```
def imprimir_elementos(lista):  
    for elemento in lista:  
        print(elemento)
```

Tiempo Cuadrático - O(n²)

```
def imprimir_pares(lista):  
    for i in lista:  
        for j in lista:  
            print(i, j)
```

Tiempo Logarítmico - O(log n)

```
def busqueda_binaria(lista, objetivo):
    bajo = 0
    alto = len(lista) - 1

    while bajo <= alto:
        medio = (bajo + alto) // 2
        if lista[medio] == objetivo:
            return medio
        elif lista[medio] < objetivo:
            bajo = medio + 1
        else:
            alto = medio - 1
    return -1
```

Tiempo Linealítico - $O(n \log n)$

```
def merge_sort(lista):
    if len(lista) > 1:
        medio = len(lista) // 2
        izquierda = lista[:medio]
        derecha = lista[medio:]

        merge_sort(izquierda)
        merge_sort(derecha)

        i = j = k = 0

        while i < len(izquierda) and j < len(derecha):
            if izquierda[i] < derecha[j]:
                lista[k] = izquierda[i]
                i += 1
            else:
                lista[k] = derecha[j]
                j += 1
            k += 1

        while i < len(izquierda):
            lista[k] = izquierda[i]
            i += 1
            k += 1

        while j < len(derecha):
            lista[k] = derecha[j]
            j += 1
            k += 1
```

Explicación de $O(\log n)$ y $O(n \log n)$

O(log n)

La complejidad $O(\log n)$ se refiere a algoritmos que reducen el tamaño del problema en cada paso, generalmente dividiéndolo en partes más pequeñas. Un ejemplo clásico es la **búsqueda binaria**. En cada iteración, la búsqueda binaria divide el espacio de búsqueda a la mitad, lo que significa que el número de elementos a considerar se reduce exponencialmente.

Ejemplo: Búsqueda Binaria

```
def busqueda_binaria(lista, objetivo):
    bajo = 0
    alto = len(lista) - 1

    while bajo <= alto:
        medio = (bajo + alto) // 2
        if lista[medio] == objetivo:
            return medio
        elif lista[medio] < objetivo:
            bajo = medio + 1
        else:
            alto = medio - 1
    return -1
```

En este ejemplo, cada vez que no encontramos el objetivo, descartamos la mitad de los elementos restantes, resultando en una complejidad logarítmica $O(\log n)$.

O(n log n)

La complejidad $O(n \log n)$ es común en algoritmos de ordenamiento eficientes como **Merge Sort** y **Quick Sort**. Estos algoritmos dividen el problema en subproblemas más pequeños ($\log n$ divisiones) y luego combinan las soluciones de estos subproblemas (n combinaciones).

Ejemplo: Merge Sort

```
def merge_sort(lista):
    if len(lista) > 1:
        medio = len(lista) // 2
        izquierda = lista[:medio]
        derecha = lista[medio:]

        merge_sort(izquierda)
        merge_sort(derecha)

        i = j = k = 0

        while i < len(izquierda) and j < len(derecha):
            if izquierda[i] < derecha[j]:
                lista[k] = izquierda[i]
```

```

        i += 1
    else:
        lista[k] = derecha[j]
        j += 1
    k += 1

    while i < len(izquierda):
        lista[k] = izquierda[i]
        i += 1
        k += 1

    while j < len(derecha):
        lista[k] = derecha[j]
        j += 1
        k += 1

```

En este ejemplo, el algoritmo divide la lista en dos partes (log n divisiones) y luego combina las partes ordenadas (n combinaciones), resultando en una complejidad $O(n \log n)$.

Tabla de Complejidades Temporales

Complejidad Temporal	Ejemplo de Sentencia	Descripción
$O(1)$	<code>return lista[0]</code>	Acceso directo a un elemento en una lista.
$O(\log n)$	<code>while n > 1: n = n // 2</code>	Búsqueda binaria.
$O(n)$	<code>for elemento in lista:</code> <code>print(elemento)</code>	Recorrido de una lista.
$O(n \log n)$	<code>merge_sort(lista)</code>	Algoritmo de ordenación Merge Sort.
$O(n^2)$	<code>for i in range(n): for j in</code> <code>range(n): ...</code>	Búsqueda de pares en una lista.
$O(2^n)$	<code>def fibonacci(n): if n <= 1:</code> <code>return n ...</code>	Algoritmo recursivo para calcular el Fibonacci.
$O(n!)$	<code>def permutaciones(lista): ...</code>	Generación de todas las permutaciones posibles de una lista.

Esta tabla resume las diferentes complejidades temporales comunes y proporciona ejemplos de sentencias que tienen esas complejidades.

Ejemplo Completo de Cálculo de Complejidad Temporal

Consideremos una función recursiva que realiza una búsqueda en una lista de listas y calcula la suma de los elementos encontrados. Esta función tiene dos bucles `for` y realiza una búsqueda lineal en cada llamada recursiva.

```
def buscar_y_sumar(lista_de_listas, objetivo):  
    suma = 0  
    for lista in lista_de_listas:  
        for elemento in lista:  
            if elemento == objetivo:  
                suma += elemento  
    if len(lista_de_listas) > 1:  
        medio = len(lista_de_listas) // 2  
        suma += buscar_y_sumar(lista_de_listas[:medio], objetivo)  
        suma += buscar_y_sumar(lista_de_listas[medio:], objetivo)  
    return suma
```

Análisis de Complejidad Temporal

1. **Bucle Externo (for lista in lista_de_listas):** Este bucle se ejecuta n veces, donde n es el número de listas en `lista_de_listas`.
2. **Bucle Interno (for elemento in lista):** Este bucle se ejecuta m veces en promedio, donde m es el número de elementos en cada lista.
3. **Llamadas Recursivas:** La función se llama recursivamente dos veces, dividiendo `lista_de_listas` en dos partes en cada llamada.

La complejidad temporal de los bucles anidados es $O(n * m)$. La recursión divide el problema en dos subproblemas de tamaño $n/2$ en cada llamada, resultando en una complejidad adicional de $O(\log n)$.

Combinando estos factores, la complejidad temporal total de la función es $O(n * m * \log n)$.